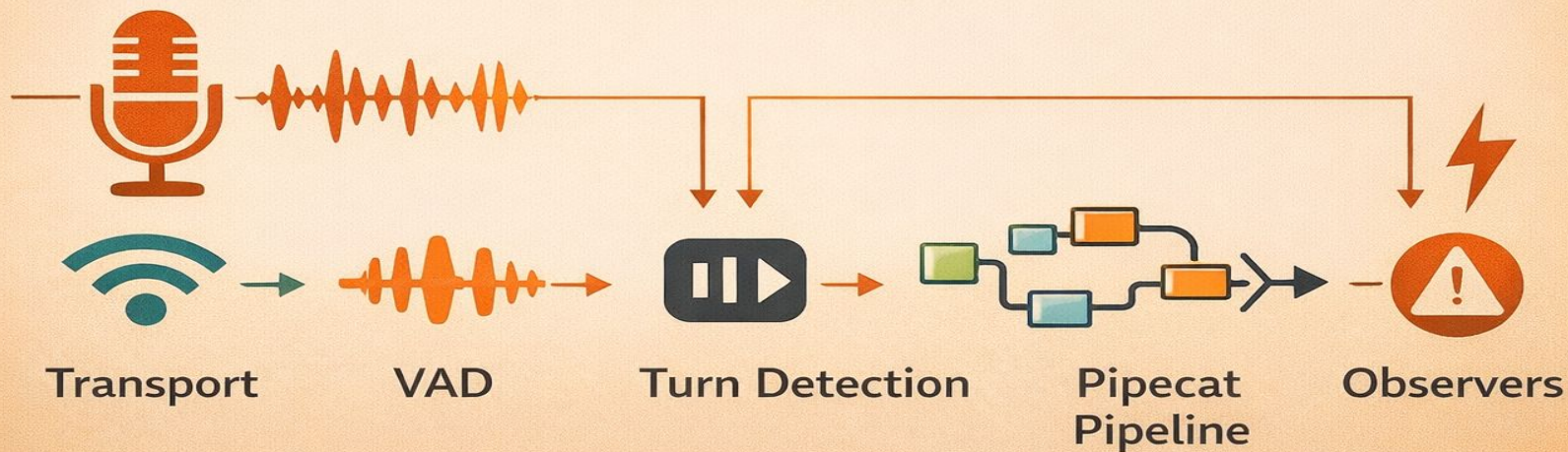


Pipecat Explained

Real-time Voice AI

— Kushagra Jain —



— What Is a Voice AI Agent?

A Voice AI agent is a system that can:

- Listen to human speech continuously
- Understand meaning
- Reason about a response
- Speak back naturally
- Allow interruptions (like humans do)

This is fundamentally different from:

“Record → upload → wait → reply” systems

Voice AI must feel like a phone call, not a chatbot.

— Why Real-Time Is Non-Negotiable

Humans expect:

- Immediate feedback
- Overlapping speech
- Natural pauses
- Interruptions

If latency > ~500ms:
Conversations feel broken
Users stop trusting the agent

→ HTTP is not enough
→ WebRTC is required.

— What Is WebRTC (aiortc)?

Conceptually

WebRTC is a technology for:

- Low-latency
- Encrypted
- Real-time audio/video streaming

aiortc

aiortc is WebRTC for Python. It:

- Handles networking (NATs, firewalls)
- Sends & receives live audio
- Does NOT do AI, STT, or TTS

**Think of aiortc as:
A real-time audio cable
between the user and Python**

Minimal aiortc mental code:

```
pc = RTCPeerConnection()  
pc.addTrack(AudioTrack())
```

That's it — aiortc just moves audio.

— What Is Pipecat?

Pipecat is a real-time voice AI framework.

It:

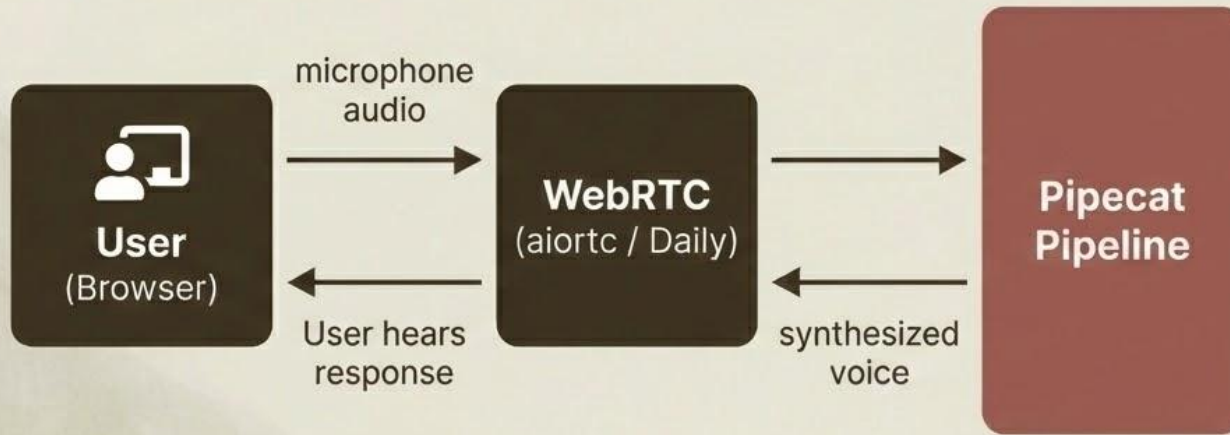
- Orchestrates audio processing
- Connects STT → LLM → TTS
- Handles streaming responses
- Handles interruptions (barge-in)
- Manages conversation flow

Pipecat does not:

- Handle WebRTC itself
- Provide a UI
- Replace LLM/STT/TTS providers

**Think of Pipecat as:
The brain and nervous system
of a voice agent.**

— High-Level Architecture



Two responsibilities:

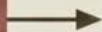
-  **Transport** → move audio
-  **Pipeline** → decide what to do with audio

— Frames (Pipecat's Core Abstraction)

Everything in Pipecat is a Frame.

Examples:

AudioFrame



audio samples

TextFrame



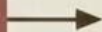
text

LLMRunFrame



“run the LLM now”

Control frames



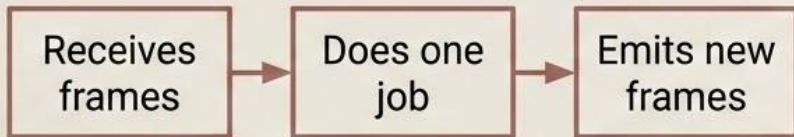
stop, interrupt, etc.

This allows:

- Streaming
- Partial results
- Real-time control

— What is a Processor?

A **Processor** is a single step in a Pipecat pipeline that:



Processors are chained together to form a pipeline.

Why Processors Matter

- Modular & replaceable
- Stream-friendly
- Easy to extend
- Clear separation of concerns

Examples of Processors



STT Processor

Audio → Text



LLM Processor

Text → Reasoning → Text



TTS Processor

Text → Audio



RTVI Processor

Controls timing, streaming, interruptions



Context Aggregator

Maintains conversation memory

— What a Processor REALLY Is

At a conceptual level:

A processor is a function that reacts to incoming frames and produces outgoing frames — often asynchronously and incrementally.

It **does not**:

- Handle networking
- Manage WebRTC
- Control the event loop

It only focuses on **one** responsibility.

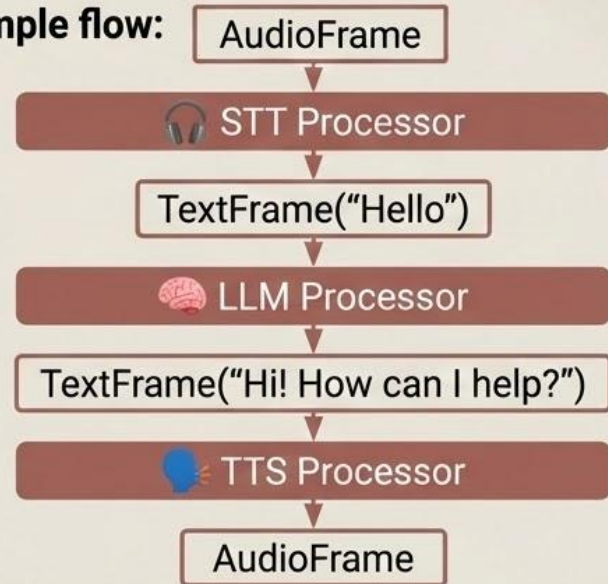
This is why Pipecat can:

- Stream partial responses
- Interrupt mid-sentence
- Cancel downstream processing

Processors Work with Frames

Processors don't talk directly to each other. They communicate using frames.

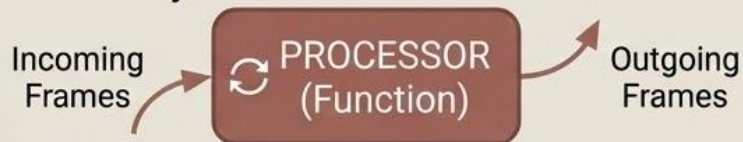
Example flow:



— What a Processor REALLY Is

At a conceptual level:

A processor is a function that reacts to incoming frames and produces outgoing frames — often asynchronously and incrementally.



It only focuses on **one** responsibility.

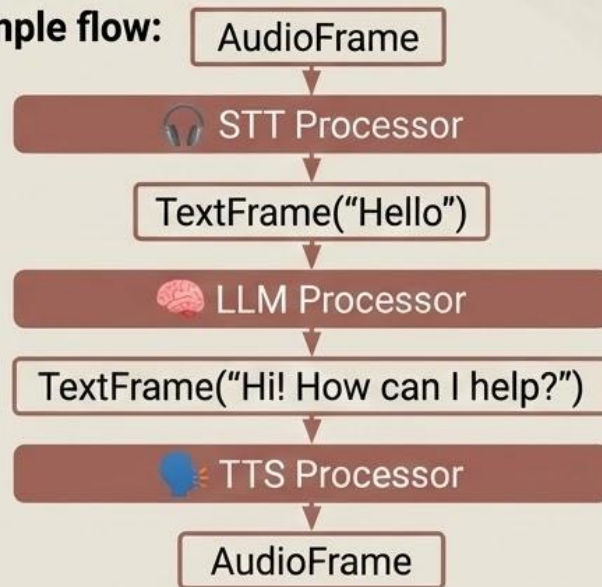
This is why Pipecat can:



Processors Work with Frames

Processors don't talk directly to each other. They communicate using frames.

Example flow:



— VAD (Voice Activity Detection)

What problem does VAD solve? “Is the human speaking right now?”

VAD:

- Detects speech vs silence
- Prevents sending silence to STT
- Reduces cost
- Improves latency

What VAD does NOT do

- Doesn't understand meaning
- Doesn't decide turns
- Doesn't control the AI

– Turn Analyzer

What problem does it solve? “Is the human done speaking?”

The Human Challenge:

Humans pause mid-sentence:
“So I was thinking... uh...
maybe...”

The Turn Analyzer:

- Waits intelligently
- Prevents cutting users off
- Signals when a turn ends.

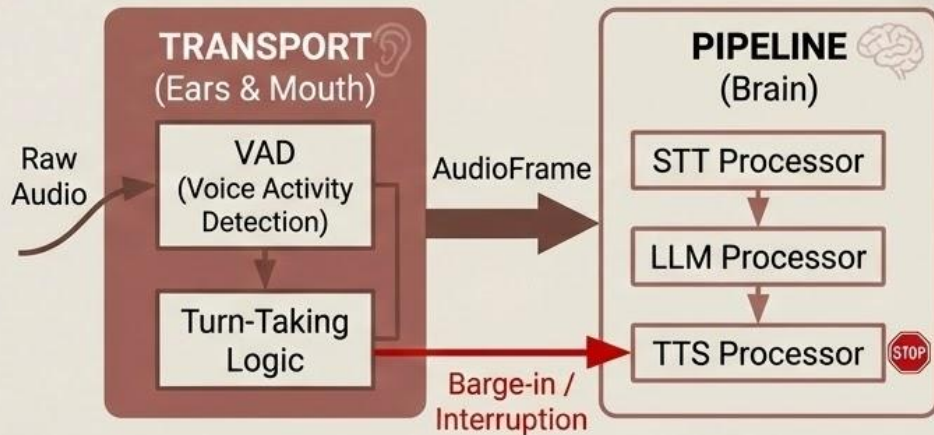
— Why VAD & Turn-Taking Run Inside the Transport (Before Audio Hits the Pipeline)

- Work directly on raw audio
- Must react in milliseconds
- Enable instant interruption (barge-in)
- Prevent wasted STT / LLM / TTS work
- Keep the pipeline meaning-level only

Mental Model

Transport = ears and mouth

Pipeline = brain



— The Pipecat Pipeline Concept

A Pipeline is a chain of processors.

Each step:

- Receives frames
- Emits frames

This makes the system:

- Modular
- Replaceable
- Streamable

Typical voice pipeline



— Pipeline Order (Why It Matters)

```
Pipeline([  
    transport.input(),    # audio from user  
    rtvi,                 # coordination  
    stt,                  # audio → text  
    llm,                  # thinking  
    tts,                  # text → audio  
    transport.output(),   # audio to user  
])
```

RTVI must come early so it can:

- Observe input
- Control downstream behavior
- Inject control frames

— Pipeline vs PipelineTask vs PipelineRunner

Pipeline

- Blueprint of what should happen
- Ordered list of processors
- No runtime state

PipelineTask

- One live conversation
- Holds state, observers, metrics
- Can be cancelled or restarted

PipelineRunner

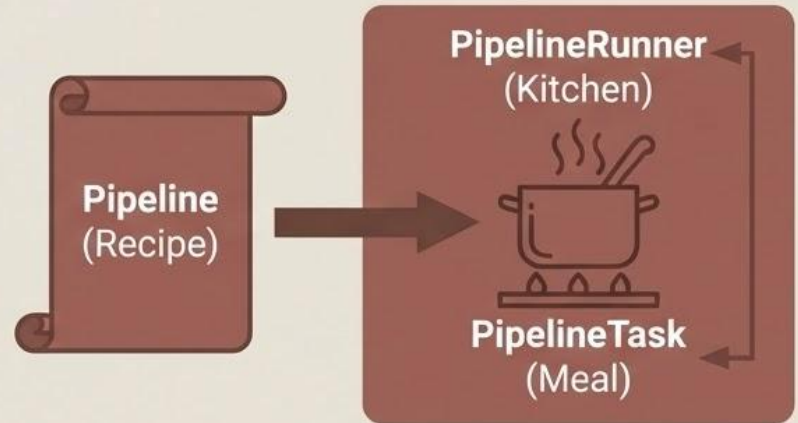
- Executes tasks
- Manages async loop & shutdown

Mental Model

Pipeline = recipe

PipelineTask = meal being cooked

PipelineRunner = kitchen



— Minimal Pipecat Code Example

```
pipeline = Pipeline([
    transport.input(),
    rtvi,
    stt,
    llm,
    tts,
    transport.output(),
])
task = PipelineTask(pipeline)
runner = PipelineRunner()

await runner.run(task)
```

This runs a live, real-time voice agent.

— RTVI (The Most Confusing Part)

RTVI $\neq$ VAD $\neq$ Turn Analyzer

This is the key insight:

**VAD & Turn Analyzer
listen to the human**

**RTVI controls the AI's
behavior**

RTVI decides:

- When the LLM should run
- Whether to stream partial responses
- Whether to interrupt TTS
- Whether to cancel an LLM mid-response

Example: Barge-In

AI is speaking: "Sure, I can help you with—"
User interrupts: "Actually, wait..."

Component	Action
VAD:	Detects speech
Turn Analyzer:	Not involved yet
RTVI:	Cancels TTS & LLM immediately

Without RTVI: AI keeps talking. Conversation feels broken.

— Observers in Pipecat

Slide Content

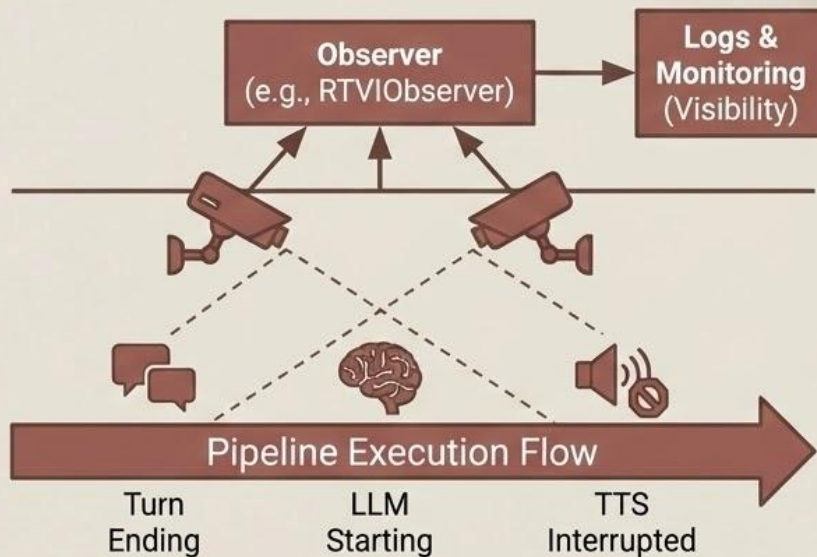
- Watch what happens during execution
- Receive important events only
- Do NOT change pipeline behavior
- Used for visibility and coordination

Example: RTVIObserver

- Example: RTVIObserver

Mental Model

Observers = security cameras



— RTVI vs RTVIObserve

Slide Content

RTVI (Processor)

- Lives inside the pipeline
- Can cancel LLM or TTS
- Injects control frames
- Actively changes behavior

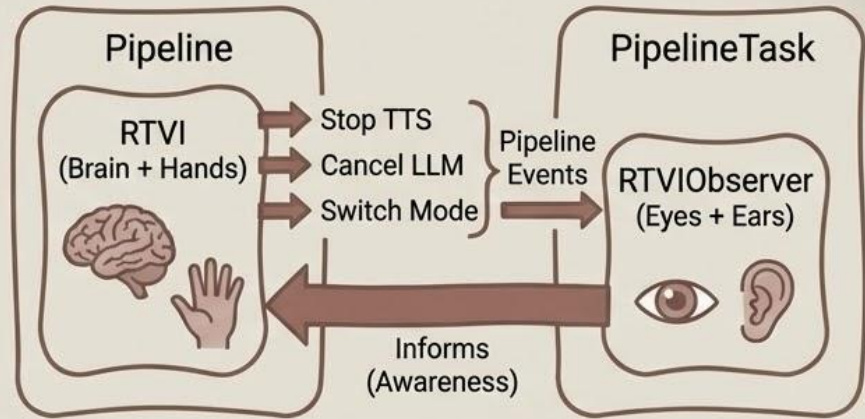
RTVIObserver

- Lives in PipelineTask
- Listens to pipeline events
- Informs RTVI what actually happened

Mental Model

RTVI = brain + hands

RTVIObserver = eyes + ears



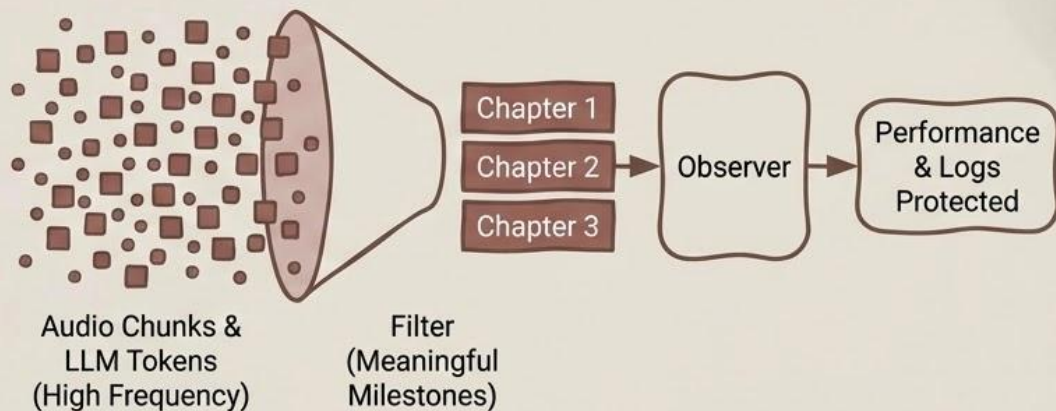
— Do Observers Get Notified at Every Step?

Slide Content

- ✗ Not for every audio chunk
- ✗ Not for every LLM token
- ✓ Only meaningful milestones
 - Protects real-time performance

Mental Model

Observers see chapters, not every word



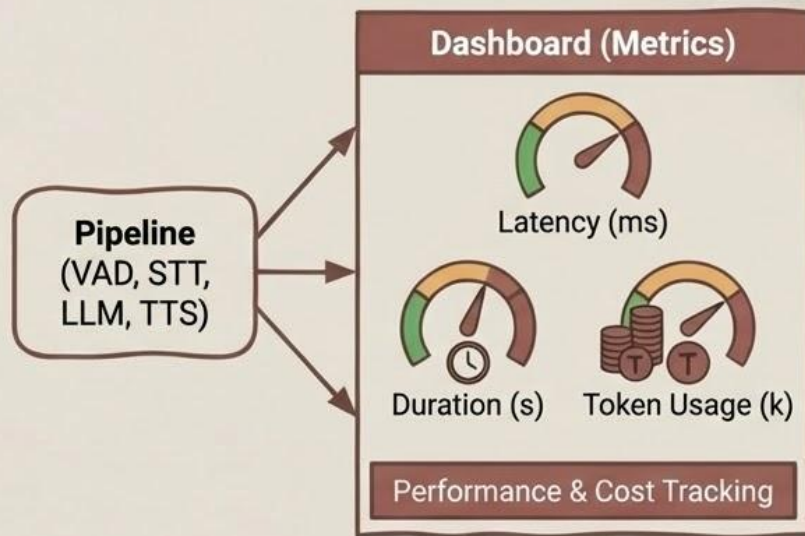
— Metrics in Pipecat

Slide Content

- Numbers collected per conversation
- Latency, duration, token usage
- Enabled per PipelineTask
- Used for cost and performance tracking

Mental Model

Metrics = dashboard gauges



— What Happens When a User Connects?

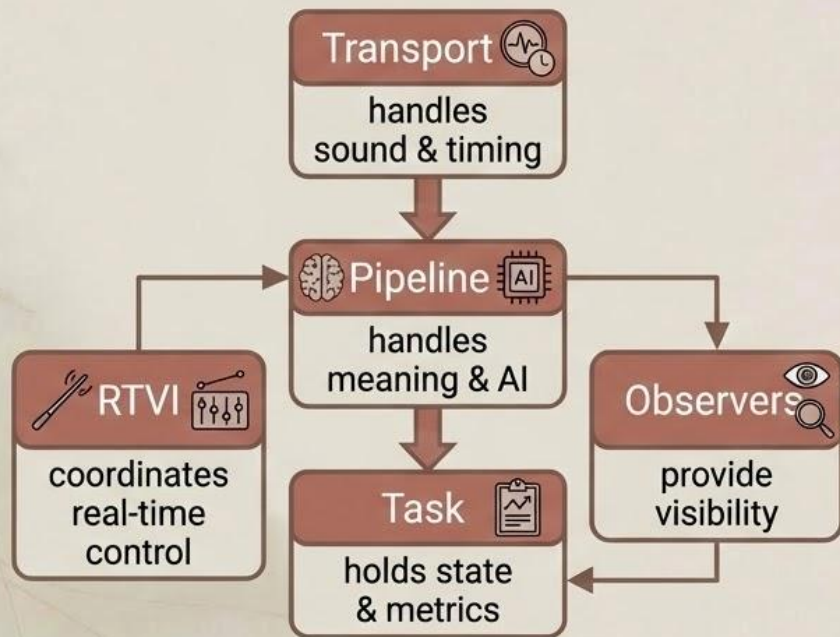
```
@transport.event_handler("on_client_connected")
async def on_connect(_, __):
    await task.queue_frames([LLMRunFrame()])
```

This:

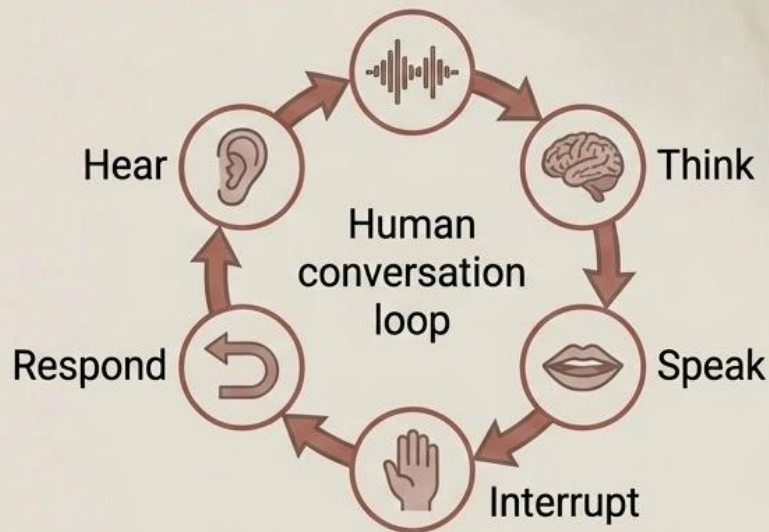
- Triggers the LLM
- Makes the bot speak first
- Starts the conversation naturally

— Pipecat in One Mental Picture

Slide Content



Mental Model



Pipecat mirrors how humans converse:
Hear → think → speak → interrupt → respond.
That's why it feels natural in real-time voice.

— Putting It All Together (Final Mental Model)

- aiortc moves sound
- VAD detects speech
- Turn Analyzer detects turn completion
- RTVI controls real-time AI behavior
- Pipecat Pipeline transforms audio → intelligence → audio

Result:



Natural



Low latency



Interruptible



Conversational

— **Final One-Sentence Summary (Perfect Closing Slide)**

Pipecat lets you build human-like voice conversations by combining real-time audio transport with intelligent, interruptible AI pipelines.